cleaner approach than dealing with the uploaded file yourself is to let your operating system do it by using its temp directory for uploads. The Python `tempfiles` module has a function that returns the path of your system's temp directory, so you don't have to figure that out yourself.

The following listing contains a new test method that creates a `Venue Model`, this time including an uploaded file. In real life (as opposed to "book life"), this code would be included as the creation part of `test_edit_venue()`, as it also tests the other fields, but for learning purposes, I've separated it out into its own thing.

Listing 8.10 Test Venue creation, including a file upload

```
# bands/tests.py
from base64 import b64decode
from django.core.files.uploadedfile import SimpleUploadedFile
...
class TestBands(TestCase):
    ...
    def setUp(self):
        ...
        # Base64 encoded version of a single pixel
        # GIF image
        self.image = "R0lGODdhAQABAIABAP///wAAACwAAAAAAQABAAACAkQBADs="
        self.image = b64decode(image)

    @override_settings(
        MEDIA_ROOT=tempfile.gettempdir())
    def test_edit_venue_picture(self):

        file = SimpleUploadedFile("test.gif", self.image)
        data = {
            "name": "Name",
            "description": "Description",
            "picture": file,
        }

        self.client.login(username="owner", password=self.PASSWORD)
        url = "/bands/edit_venue/0/"
        response = self.client.post(url, data)

        self.assertEqual(302, response.status_code)
        venue = Venue.objects.first()
        self.assertIsNotNone(venue.picture)
```

Decode the binary representation of a GIF from the preceding base64-encoded string.

Override the MEDIA_ROOT configuration value to point to the operating system's temp directory, so any uploaded files get cleaned up.

Create an in-memory file for testing based on the GIF binary data.

Like in previous tests, check the object got created, this time validating the .picture attribute.

There it is—that's the complete picture (OK, there are bad puns, and then there are bad puns I shouldn't be allowed to get away with). You've got the tools now to create tests for all the views you've written so far.

8.7 More testing techniques

The start of this chapter introduced the testing pyramid and some terms used in the programming industry. Everything so far falls into the unit test and integration test levels. You can test Django projects at the UI level as well, but it requires a bit more work.